

Vibecodear, o hacer <cosas> con palabras // vía LLMs

Taller de uso avanzado de IA · tercera reunión

Letra y Código — Departamento de Letras, FFyL, UBA

Mauro Santelli

Julio de 2026

Universidad de Buenos Aires · IIF-SADAF (CONICET)

Introducción

1. ¿Quiénes usaron un chat de IA para *escribir* algo?

Introducción

1. ¿Quiénes usaron un chat de IA para *escribir* algo?
2. ¿Quiénes usaron una terminal alguna vez?

Hoy

1. **Concepto y demo mínima**

Qué es el vibecoding, qué es un programa, las formas de hacerlo.

2. **Construcción en vivo**

Hacemos la página de recursos de este taller, de la especificación al resultado.

3. **Publicar y cerrar**

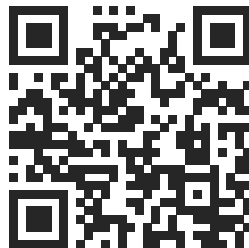
La página queda en internet. Mapa de lo que no vimos, límites, costos.

Al final de la clase, la página existe y es pública.

Una regla para toda la clase

Cada vez que aparezca una palabra que no entienden, mándenla a este formulario. Anónimo, desde el teléfono, sin esperar ni pedir permiso. Pueden ir varias juntas, separadas por comas.

Al final de la clase, esas palabras van a estar explicadas en la página que vamos a construir — publicadas y con sus dudas como fuente.



forms.gle/n6gDQ4CBMEgvyLWZ8

Vibecoding

Un mal nombre para una familia de prácticas

Codear: escribir código en un lenguaje artificial para que una máquina ejecute funciones.

Vibe: vibras.

Vibecoding, entonces: codear por vibras.

De lo que hablamos: **delegar la escritura del código a un modelo de lenguaje**. Programar con mayor o menor asistencia. Cuando la asistencia es mucha, lo llamamos vibecoding.

La implicatura del nombre

El nombre sugiere que uno *tropieza* con el resultado sin destreza o habilidad en juego.

La destreza está en otro lado:

- destreza de **especificador** — decir bien qué se quiere,
- destreza de **evaluador** — juzgar bien lo que el modelo devuelve.

Hacer cosas con palabras

Un programa es texto que un intérprete ejecuta.

Un prompt es texto que produce ese texto.

Escribir un prompt es dar una directiva; que salga bien depende de que el intérprete entienda eso que uno quiso pedir. El problema central del vibecoding es un problema de **uptake** — y el intérprete, esta vez, no es humano.

Programas

La versión mínima



Un programa es un texto escrito en un lenguaje artificial que una computadora lee y ejecuta, en general secuencialmente: una instrucción tras otra.

Nombres que van a aparecer: **Python**, **JavaScript**, **C**, **HTML**. Para lo que viene alcanza con distinguir qué *tipo* de texto es cada uno.

HTML no es programación: es marcado

Markdown

Recursos del taller

Esta página reúne los
****materiales**** del curso.

Texto con etiquetas. Anotar textos es una práctica vieja: crítica textual, corrección de pruebas.
Esto es lo mismo, con otra convención.

Un archivo HTML se abre en el navegador con doble click: con eso ya existe una página.

HTML

<h1>Recursos del taller</h1>

<p>Esta página reúne los
materiales
del curso.</p>

Las formas del vibecoding

Cuatro maneras de pedirle código a un modelo. Todas hacen lo mismo —producir código a partir de castellano— con distinta fricción y distinto alcance:

1. **Chat en la web** (ChatGPT, Claude, Gemini)
2. **Agentes en la web** (Claude Code web, Codex, AI Studio)
3. **Chat en el editor** (VS Code con Claude, Antigravity, Cursor)
4. **Agentes en la terminal** (Claude Code, Codex CLI)

El orden va de menor a mayor compromiso — y de menor a mayor alcance.

1 · Chat en la web

Se le pide el código a un chat y el resultado se copia a mano a un archivo. El modelo no ve la computadora: solo ve la conversación.

A favor — no hay que instalar nada; alcanza con el plan gratuito; se aprende leyendo el código que devuelve.

En contra — el traslado es manual (copiar, pegar, guardar, abrir); el modelo no ve ni los archivos ni el resultado, así que cada corrección hay que contársela; con más de un archivo se vuelve inmanejable.

2 . Agentes en la web

El agente corre en una computadora del proveedor: edita archivos, ejecuta y muestra el resultado, todo dentro del navegador.

A favor — tampoco hay que instalar nada; el agente sí ve y edita los archivos; vista previa integrada; puede exportar el proyecto a GitHub.

En contra — el trabajo vive en la nube de un proveedor; poco control sobre el entorno; conectar herramientas propias es difícil.

3 · Chat en el editor

El editor de código de siempre, con un panel lateral de chat que lee y escribe los archivos del proyecto, en la máquina propia.

A favor — los archivos son locales; cada cambio se ve como una diferencia que se acepta o rechaza; combina editar a mano y pedir.

En contra — hay que instalar y configurar; la interfaz del editor tiene su propia curva de aprendizaje; el agente ejecuta menos por su cuenta.

4 . Agentes en la terminal

El agente vive en la línea de comandos, con permiso para leer, escribir y *ejecutar*. Puede usar cualquier programa y delegar trabajo en subagentes.

A favor — alcance máximo: git, APIs, herramientas de todo tipo; automatiza de punta a punta; es lo que presuponen los tutoriales serios.

En contra — la terminal intimida al principio; requiere instalación y, según el caso, suscripción; más poder exige más cuidado (permisos, aislamiento).

Las cuatro, en resumen

	Chat web	Agente web	Editor	Terminal
Hay que instalar	no	no	sí	sí
Ve los archivos	no	remotos	locales	locales
Ejecuta y prueba solo	no	sí	a veces	sí
Iteración	manual	fluida	fluida	fluida
Ideal para	piezas chicas	empezar	trabajo con control	proyectos completos

La demo que sigue enfrenta la primera con la última: el mismo pedido, dos fricciones.

Demo mínima: el mismo pedido, dos fricciones

El pedido, fijo e idéntico en las dos:

«Una página HTML, un solo archivo, con el poema Ajedrez de Borges, tipografía serif grande, fondo claro; al hacer click sobre el poema los colores de fondo y texto cambian a una paleta distinta, rotando entre tres paletas.»»

1. Primero en un chat común, con copiar y pegar.
2. Después con un agente en la terminal.

Especificación

El 80 % del trabajo es decir bien lo que uno quiere

Escribir un pedido concentra problemas clásicos del estudio del lenguaje, ahora con consecuencias inmediatas:

- **Ambigüedad** — ¿“una sección por clase” es una sección *para* cada clase o *sobre* cada clase?
- **Vaguedad** — ¿qué cuenta como “una página linda”?
- **Presuposiciones** — lo que uno da por sabido y el modelo no sabe.
- **Conocimiento compartido** — el modelo no estuvo en las dos reuniones anteriores.

La formación en lenguaje es una **ventaja comparativa** para programar así.

Prompts malos que igual funcionan

Cuatro pedidos que cualquier chat convierte en una página que se abre y funciona:

1. «Hacé una página linda sobre poesía.»
2. «Una página con un poema corto de Borges en inglés.»
3. «Hacé la página para el taller.»
4. «Una página minimalista, con muchos colores y animaciones por todos lados, pero sobria.»

Ninguno falla.

Se pueden probar en cualquier chat: pedir el resultado «como un solo archivo HTML», guardarlo como `pagina.html`, abrir con doble click.

Qué decidió el modelo por su cuenta

1. **Vaguedad** — «linda», «sobre poesía»: todas las decisiones quedaron delegadas.
2. **Ambigüedad** — ¿un poema que Borges *escribió* en inglés, o *traducido* al inglés? El modelo elige sin avisar, y puede inventar la traducción.
3. **Presuposición** — ¿qué taller? El modelo no estuvo en las reuniones: rellena con un taller genérico.
4. **Contradicción** — minimalista y recargada a la vez. El modelo no protesta: promedia.

Un mal prompt es un **borrador**: la iteración empieza por ver qué decidió el modelo sin nosotros.

Demo: pedido vago contra pedido especificado

1. *“Hacé una página linda sobre el taller.”*
2. `spec.md`: la especificación real, escrita de antemano (viene enseguida).

Comparamos los dos resultados en vivo.

En vivo

El punto de partida: una carpeta preparada

La carpeta ya está lista desde antes de la clase. Adentro hay tres cosas.

```
recursos-taller/
```

<code>spec.md</code>	qué página queremos, sección por sección
<code>recursos.md</code>	los enlaces de las tres reuniones, ya curados
<code>materiales/</code>	textos e imágenes que la página va a usar

El agente va a leer todo esto. Preparar los insumos —elegir, ordenar, nombrar— ya es **especificar**.

spec.md, **abreviada**

## Objetivo	repaso del taller, para gente de letras
## Modo de trabajo	sin preguntas: lo no fijado es del agente
## Requisitos	Vercel · modo oscuro · código comentado
## Materiales	integrar materiales/ y recursos.md
## Estética	serif editorial + mono de terminal
## Estructura	módulos · glosario · seguridad · cómo se hizo

Tres frases textuales:

- «Lo único que importa es que funcione bien.»
- «El trabajo principal lo hace la tipografía.»
- «Que parezca hecha con criterio, no con plantilla.»

La instrucción inicial, completa

«Leé `spec.md` y construí el sitio que describe. Cada vez que te pida un cambio a partir de ahora, agregá el prompt textual a la lista “Iteraciones” de la sección “Cómo se hizo esta página”.»

Dos oraciones. La segunda deja la página **autodocumentada**: cada prompt de la clase queda registrado en la página misma.

El primer resultado es un borrador

Regla general: lo primero que sale es un borrador. Se critica y se itera.

Tres cambios, en orden:

1. De forma (tipografía, modo oscuro).
2. De estructura (orden de secciones, navegación).
3. De contenido (retocar un párrafo introductorio): texto y código se editan igual.

Publicar

Dos ideas mínimas

Repositorio: guardar versiones con su historial. Cada estado del proyecto queda registrado y se puede volver atrás.

Deploy: poner la carpeta en una computadora que está siempre prendida, con una dirección pública.

Con eso alcanza por hoy. Herramientas: `git`, GitHub, Vercel (alternativas: Cloudflare, Netlify).

Demo: de la carpeta a internet — los comandos

El agente los escribe y los ejecuta. Qué hace cada uno:

1. `git init` — la carpeta se vuelve repositorio: aparece un historial, por ahora vacío.
2. `git add .` — selecciona todo el contenido de la carpeta (el punto significa «acá») para la próxima foto.
3. `git commit -m "versión inicial"` — saca la foto: un estado completo, con autor, fecha y mensaje.
4. `gh repo create recursos-taller --public --source=. --push` — crea el repositorio en GitHub y sube el historial.



`github.com/msantelli/
recursos-taller`

El repositorio queda **público**: el ejemplo completo de la clase, con su historial.

Demo: de la carpeta a internet — Vercel

El último paso: entrar a `vercel.com` y apretar dos botones.

1. **Add New → Project** — importar el repositorio recién creado desde GitHub.
2. **Deploy** — Vercel lee la carpeta, publica y devuelve una **URL pública**.

De ahí en más, cada `push` al repositorio republica solo. (Esto vuelve enseguida.)

[QR con la URL de la página — abrirla en el teléfono, ahora]

Esto que acabamos de hacer ya está en internet. La página documenta cómo fue hecha, con los prompts incluidos.

Lo que acaba de pasar, mirado de cerca

En el deploy, el modelo no escribió código: escribió **comandos** —`git init`, `gh repo create`, `deploy`— y los ejecutó.

Un comando es una directiva a la computadora. Otra vez: hacer cosas con palabras.

La consecuencia: un agente en la terminal puede usar **cualquier programa de línea de comandos**. La terminal es su caja de herramientas.

Darle herramientas a un modelo: dos vías

Programas de línea de comandos (CLI). El modelo ejecuta los mismos programas que ejecutaría una persona: `git` y compañía, pero también herramientas de trabajo académico — el CLI de Obsidian para leer y buscar en las notas, la API de Zotero para consultar la biblioteca.

MCP (Model Context Protocol). Un protocolo estándar para que una aplicación exponga sus datos y acciones a cualquier modelo, sin que el modelo tenga que conocer sus comandos específicos. (Hay servidores MCP para Zotero, Obsidian, Drive, calendarios. . .)

La reunión pasada mostró qué es un agente; esto es el *cómo* concreto.

Demo: una cita desde un DOI

Una API es exactamente esto: una URL que devuelve texto estructurado. El pedido:

```
curl https://api.crossref.org/works/10.1093/mind/IV.14.278
```

La respuesta, recortada (la real trae unos 35 campos):

```
{ "status": "ok",  
  "message": {  
    "title": ["WHAT THE TORTOISE SAID TO ACHILLES"],  
    "author": [{ "given": "LEWIS", "family": "CARROLL" }],  
    "container-title": ["Mind"],  
    "volume": "IV", "issue": "14", "page": "278-280",  
    "published-print": { "date-parts": [[1895]] },  
    "is-referenced-by-count": 544,  
    "URL": "https://doi.org/10.1093/mind/iv.14.278" } }
```

Lo que el agente produce con eso

El prompt pide: consultá esa URL y agregá la referencia a la página, en una sección nueva «Para leer más». El resultado:

Carroll, L. (1895). «What the Tortoise Said to Achilles». Mind, IV(14), 278–280.
<https://doi.org/10.1093/mind/IV.14.278>

- El título llegó EN MAYÚSCULAS: los datos vienen **crudos**, y formatearlos es exactamente el trabajo que delegamos.
- La cita sale de la fuente: el modelo la **consulta** y la **edita** — lo contrario de alucinarla.
- Nadie tipeó la cita. El mismo gesto, repetido, arma una bibliografía entera.

Del formulario a la página

Queda ver si entraron más sugerencias al glosario y las usamos para actualizar la página: lo que se juntó en el formulario entra a la página. El agente hace todo con un sólo *prompt*:

1. **Descarga** la planilla, separa los términos y une duplicados.
2. **Filtra** con los criterios escritos en el pedido: **agregar** / **ya está** / **descartar**, con razones.
3. **Escribe** las entradas, commit, push: la página publicada se actualiza sola.

Cuando vamos a nuestro teléfono o entramos por web, la página ya está actualizada.

Nuestro **archivo de especificación o prompt** guía al modelo sobre qué filtrar.



Qué más se puede hacer

Una página web es solo el primer ejemplo. Cosas reales hechas del mismo modo:

- **Herramientas pedagógicas interactivas** — derivaciones lógicas, interpretación radical, diagramas sintácticos.
- **Diapositivas y documentos** — estas mismas diapositivas (LaTeX) se hicieron así.
- **Scripts de trabajo** — convertir archivos por lotes, extraer texto de PDFs, armar bibliografías desde un DOI.
- **Análisis de textos** — conteos, concordancias, corpus pequeños.
- **Chatbots y agentes propios** — incluso servidores MCP para herramientas académicas.

Los ejemplos quedan enlazados en la página que acabamos de publicar.

Mapa de lo que no vimos

Para saber qué buscar cuando haga falta:

- **Frontend** — lo que se ve en el navegador. Nuestra página es solo esto.
- **Backend** — un programa que corre del otro lado y responde pedidos.
- **Base de datos** — donde un backend guarda información que cambia.
- **Página estática** — la nuestra: archivos fijos, sin backend ni base de datos. Por eso publicarla es tan barato y simple.

Regla práctica: empezar con un sitio estático; agregar backend solo cuando algo lo exija.

Qué es gratis y qué no

- Los chats tienen planes gratuitos con límites de uso.
- Los agentes de terminal suelen requerir suscripción o pago por uso (API).
- GitHub y el plan *hobby* de Vercel son gratuitos: todo lo que publicamos hoy cuesta cero.

Tarea opcional, sin instalar nada: rehacer la página, en versión propia, con cualquier chat en la web — o partir de los prompts malos ya vistos y mejorarlos de a un defecto por vez. La spec y el resultado quedan disponibles.

Cierre

Local contra público

Una página abierta con doble click, o servida en `localhost`, existe solo en esa máquina. Publicada, la ve cualquiera — y lo publicado puede quedar cacheado aunque después se borre.

- El código se publica; las claves no. Las claves (de APIs, de servicios) viven en **variables de entorno**: un archivo `.env` que no entra al repositorio; en el hosting se cargan aparte.
- Una clave commiteada queda en el *historial* del repositorio, aunque se borre en la versión siguiente.
- Datos personales (propios o ajenos) tampoco entran a un repo público.

Nuestra página es estática y sin claves: por eso publicarla fue trivial. Con backend o APIs pagas, esto pasa a ser la primera preocupación.

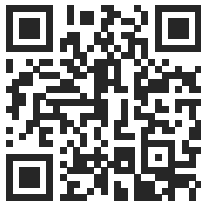
Vibecoding responsable

- El modelo **alucina código como alucina citas**. El código generado se lee —o se hace leer por otro modelo— antes de confiar en él.
- Nunca pegar claves, contraseñas ni datos personales en un prompt.
- El resultado se verifica ejecutándolo.

Trabajar con modelos de lenguaje requiere el mismo cuidado que trabajar con fuentes (chequear que existan) o con colegas (chequear que hayan hecho lo que dijeron que harían).

Cierre

- Vibecodear es hacer cosas con palabras: especificar bien y evaluar bien.
- La interfaz es el castellano. La escritura cuidadosa es la habilidad técnica.
- Hoy: una página real, pública, que documenta su propio método.



`recursos-taller-llms.vercel.app`